arXiv:2509.25020v2 [cs.LG] 2 May 2026

Deep Thinking by Markov Chain of Continuous Thoughts

Jiayu Liu [1 †] **Zhenya Huang** [1] **Xuan Yang** [2 3] **Tianyun Ji** [1] **Anya Sims** [4] **Hao Xu** [5] **Enhong Chen** [1]
Yee Whye Teh [4] **Ning Miao** [2 3]

Resumen

Los modelos basados en transformadores pueden realizar razonamientos complejos generando caminos de razonamiento token por token. Si bien este enfoque es efectivo, a menudo requiere generar miles de tokens para resolver un solo problema, lo que puede ser lento y costoso en términos computacionales. Más importante aún, implica una operación de muestreo discreto al final de cada paso de tiempo, creando un cuello de botella de información a lo largo del tiempo. En este trabajo, proponemos MarCos, una mejora en la estructura del transformador que permite un razonamiento completamente continuo a nivel de pensamiento. A diferencia de las capas tradicionales de transformador, que se centran en refinar las predicciones de tokens en cada paso de tiempo, las capas de MarCos mapean una representación continua de un pensamiento gradual a la distribución del siguiente pensamiento. Esto nos permite lograr un razonamiento de múltiples pasos en una sola pasada de MarCos. Los resultados experimentales preliminares en tareas matemáticas sintéticas y del mundo real demuestran el gran potencial de MarCos. Notablemente, observamos que el mayor ancho de banda de información de MarCos estimula la capacidad de pensamiento paralelo, en contraste con el pensamiento de un solo hilo en los transformadores tradicionales. Además, en tareas matemáticas del mundo real, MarCos logra más de 10 veces más velocidad en tiempo real, manteniendo el mismo nivel de precisión. Nuestro código está disponible en https://github.com/Ljyustc/MarCos.

1. Introducción

Los modelos basados en transformadores demuestran sólidas capacidades de razonamiento al generar de forma autoregresiva las vías de razonamiento, token por token (Wei et al., 2022). A pesar de su simplicidad y aplicación…

†Trabajo realizado durante la visita de Jiayu a la Universidad de Oxford. 1Laboratorio Nacional de Inteligencia Cognitiva, Universidad de Ciencia y Tecnología de China 2Departamento de Ciencia de Datos, Universidad de Hong Kong 2 Departamento de Hong Kong para la Inteligencia Artificial, Universidad de Hong Kong 2 Departamento de Estadística, Universidad de Oxford 2 Ning Miao <ningmiao@cityu.edu.hk>, Zhenya Huang <huangzhy@ustc.edu.cn>.

Preimpreso. 5 de mayo de 2026.

La capacidad de generar de manera token-por-token, esta generación autoregresiva puede conducir a una inferencia ineficiente y un comportamiento de razonamiento subóptimo. Para resolver un problema, a menudo necesitamos invocar el transformador miles de veces de forma secuencial, lo que resulta en una latencia significativa. Al mismo tiempo, los transformadores se ven obligados a pensar y hablar simultáneamente. En otras palabras, necesitan generar un token inmediatamente después de una única computación hacia adelante, lo que limita considerablemente la planificación reflexiva (Valmeekam et al., 2023; Kambhampati et al., 2024). Además, la operación discreta de selección de tokens al final de cada paso de tiempo restringe severamente el ancho de banda de información entre los pasos. Por ejemplo, la información generada en la última capa del transformador solo puede transmitirse al siguiente paso a través de los tokens seleccionados, lo que provoca una pérdida significativa de información.

Para abordar estos problemas, trabajos anteriores (Deng et al., 2023; 2024) han intentado reemplazar los tokens discretos en los transformers con tokens continuos. Por ejemplo, Coconut (Hao et al., 2024) gradualmente reemplaza bloques de tokens discretos con un número menor de tokens continuos. CoLaR (Tan et al., 2025) introduce además una función de pérdida adicional para alinear los tokens continuos con la representación de los tokens discretos en las rutas de razonamiento. Aunque estos métodos reducen el número de pasos de tiempo para llegar a una respuesta final, principalmente funcionan al comprimir múltiples tokens discretos en uno continuo, lo que significa que su razonamiento latente esencialmente sigue siendo una forma de toma de decisiones basada en tokens. Además, la mayoría de estos métodos adoptan una función de transición determinista entre los tokens continuos durante el razonamiento, lo cual es inadecuado para aprender el espacio de soluciones diverso y multimodal.

En este artículo, proponemos MarCos, una variante del transformer que soporta de forma nativa el razonamiento continuo a nivel de pensamiento. En MarCos, modelamos el proceso de razonamiento paso a paso como una cadena de Markov de pensamientos continuos. A diferencia de los transformers tradicionales, cuyos capas están encargadas de refinar la predicción del siguiente token (Lioubashevski et al., 2024), MarCos emplea capas dedicadas al pensamiento para predecir directamente la distribución de los pensamientos en el siguiente paso de razonamiento, tal como se ilustra en la Figura 1. Al representar cada pensamiento como el estado de un grupo de neuronas, MarCos "piensa" internamente actualizando los estados de las neuronas. Para observar y añadir supervisión a los pensamientos intermedios, utilizamos una capa de "hablar" separada para traducir los estados de las neuronas a lenguaje natural. Este diseño…

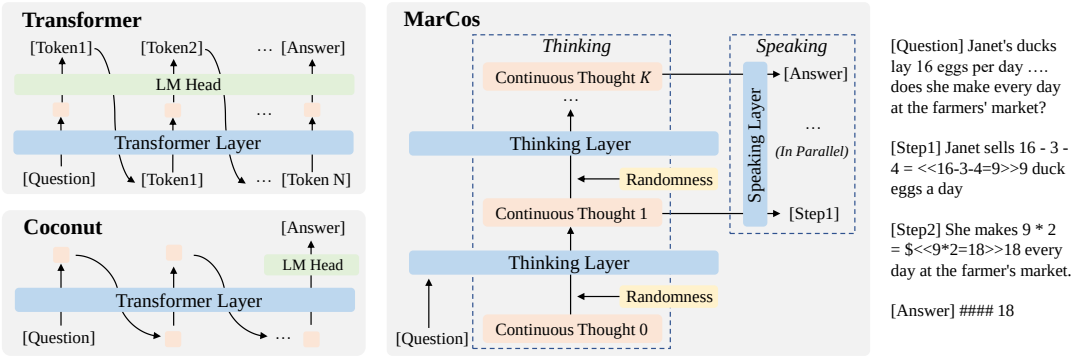


Figura 1. Comparación de un transformador tradicional, el método de razonamiento continuo representativo Coconut, y nuestro MarCos. En MarCos, el proceso de pensamiento se modela como transiciones iterativas de pensamientos continuos (de 0 a K) con incorporación de aleatoriedad. El proceso de habla (opcionalmente) traduce estos pensamientos al lenguaje natural (p. ej., [Paso 1] a [Respuesta]).

esencialmente separa el proceso de pensamiento del proceso de habla, lo que permite al modelo deliberar completamente antes de hablar. Esto también se alinea con los recientes hallazgos de la neurociencia, que sugieren que el lenguaje es principalmente una herramienta para la comunicación, más que para el pensamiento (Fedorenko et al., 2024; Fedorenko & Varley, 2016). Para modelar la diversa distribución de las posibles vías de razonamiento, incorporamos además un módulo variacional que controla explícitamente la aleatoriedad del razonamiento a nivel de paso. Con todos estos diseños, permitimos a MarCos pensar completamente en un espacio continuo, lo que crea la posibilidad de un razonamiento paralelo y más eficiente.

Inicialmente, evaluamos a MarCos en tareas sintéticas, donde descubrimos que su mayor ancho de banda permite un pensamiento paralelo, una capacidad difícil de lograr en los transformers tradicionales. Posteriormente, en tres pruebas de matemáticas a nivel de primaria (GSM8K (Cobbe et al., 2021), SVAMP (Patel et al., 2021) y MultiArith (Roy & Roth, 2015)), MarCos alcanza un rendimiento comparable o incluso superior al razonamiento basado en CoT con transformers, con una mejora de precisión del 4.7% en GSM8K y hasta un aumento de 15.7× en la velocidad de inferencia. Esto destaca el gran potencial de nuestra arquitectura. Además de la precisión, nuestro análisis revela la independencia funcional entre el procesamiento contextual y la determinación de la aleatoriedad en los módulos variacionales, y descubrimos que diferentes dimensiones de aleatoriedad controlan diferentes propiedades del habla, como la longitud de la oración y la profundidad del paso. Esto abre la posibilidad de controlar manualmente los comportamientos de razonamiento. Adicionalmente, demostramos que nuestro modelo mantiene un rendimiento competitivo al aplicar la decodificación no-autoregresiva (NAR) en la etapa del habla, lo que indica que el razonamiento y el habla desempeñan roles bien diferenciados en nuestra arquitectura. En resumen, las contribuciones de este artículo son:

- Proponemos MarCos, una nueva arquitectura que modela el razonamiento como una cadena de Markov de pensamientos continuos y que opera en etapas separadas de pensamiento y expresión.
- Presentamos una forma sistemática de modelar la aleatoriedad en el razonamiento, lo que produce dos avances clave: (1) permitiendo un control explícito sobre el proceso de razonamiento, y (2) verificando la viabilidad de predefinir la aleatoriedad a nivel de paso antes de la generación de tokens.
- Los experimentos con tareas sintéticas y matemáticas demuestran el gran potencial de MarCos, que puede lograr hasta 15.7 veces más rápido en la inferencia, sin comprometer la precisión.

2. Trabajo relacionado

Diversos métodos de razonamiento continuo han surgido para mitigar los problemas asociados con el razonamiento paso a paso, que pueden clasificarse en enfoques basados en el eje temporal y en la profundidad.

Razonamiento latente a lo largo del eje temporal. La mayoría de los enfoques basados en el eje temporal buscan mejorar la eficiencia de la inferencia al comprimir los tokens discretos en CoT explícito en unos pocos pasos continuos. Por ejemplo, iCoT-KD (Deng et al., 2023; 2024) destila toda la trayectoria de razonamiento en un solo paso de inferencia, mientras que el aprendizaje por niveles (curriculum learning) se utiliza en iCoT-SI (Deng et al., 2024) para fomentar gradualmente la transición del modelo desde las explicaciones explícitas hacia un razonamiento completamente latente. Sin embargo, comprimir toda la trayectoria de razonamiento de CoT, que puede consistir en miles de tokens, en un solo paso de inferencia es demasiado agresivo y perjudica el rendimiento del modelo. En consecuencia, los trabajos más recientes adoptan vectores continuos intermedios, cada uno de los cuales representa un bloque de tokens discretos consecutivos en el razonamiento de CoT. Por ejemplo, Coconut (Hao et al., 2024) reemplaza progresivamente bloques de tokens de palabras discretos con vectores continuos. CoLaR (Tan et al., 2025), CODI (Shen et al., 2025), SIM-CoT (Wei et al., 2025) y CoT2 (Zhang et al., 2025) además añaden supervisión para alinear los estados latentes con los bloques de tokens que reemplazan en CoT explícito, con CoT2 formando tokens continuos como combinaciones convexas de embeddings de vocabulario. Si bien mejoran la eficiencia de la inferencia, estos métodos esencialmente siguen siendo una forma de compresión de tokens y aún podrían verse limitados por el razonamiento a nivel de token. Además, sus vectores continuos se actualizan de manera fija o a través de simples transiciones gaussianas, lo que limita su capacidad para modelar el pensamiento diverso y exploratorio.

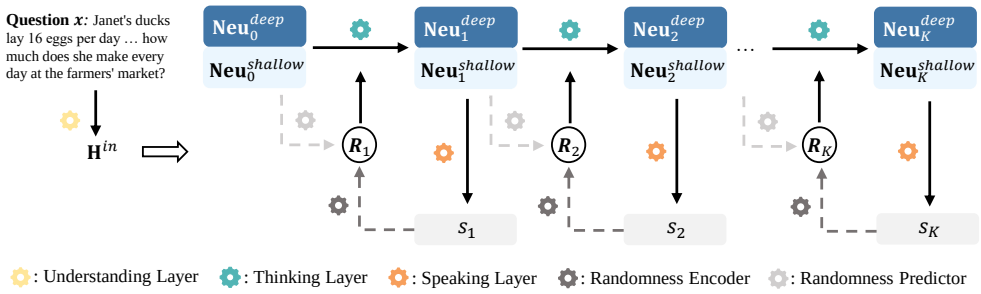


Figura 2. Ilustración de nuestro modelo MarCos. En la etapa de razonamiento, MarCos analiza en el espacio continuo durante K pasos, actualizando las neuronas Neudeepk y Neushallowk. En la etapa de expresión, {Neushallowk, k = 1, ..., K} se traduce al lenguaje natural en paralelo. Si no se requieren pasos intermedios, solo necesitamos traducir NeushallowK a una respuesta final. Durante el entrenamiento, la variable aleatoria Rk se deriva de la oración de referencia sk utilizando el codificador de aleatoriedad f, mientras que en la inferencia se muestrea de una distribución predicha por el predictor de aleatoriedad g, basado en Neudeepk, Neushallowk y Hin.

Razonamiento latente en el eje de profundidad. Estos enfoques modifican la arquitectura del transformer para soportar el razonamiento latente. Un ejemplo es el Transformer Circular (Giannou et al., 2023; Yang et al., 2024), que reutiliza los mismos parámetros de capa en cada iteración para profundizar en la deliberación interna. Por ejemplo, Geiping et al. (2025) aplican bloques de transformer de forma recurrente para cada token, lo que permite escalar en tiempo de prueba sin generar tokens adicionales. Zhu et al. (2025) incorporan además mecanismos de salida temprana adaptativos, permitiendo al modelo determinar dinámicamente el número de pasos recurrentes. En comparación con MarCos, estos métodos carecen de supervisión para los estados de razonamiento intermedio. Por lo tanto, pueden considerarse como formas eficientes en parámetros para aumentar la profundidad de los transformers, que son independientes de nuestro enfoque.

3. Nuestro modelo MarCos

En esta sección, presentamos MarCos, una nueva arquitectura para un razonamiento continuo y progresivo a nivel de pensamiento. Específicamente, modelamos el proceso de razonamiento como un modelo de Markov oculto condicional (cHMM), donde las preguntas de entrada son las condiciones, los pensamientos internos son variables ocultas, y las frases habladas son variables observables. Esta formulación garantiza una clara separación entre el pensamiento y el habla: el pensamiento corresponde a transiciones de pensamientos continuos, y el habla es el proceso de emisión de estos pensamientos al lenguaje natural. Crucialmente, el pensamiento continuo en cada paso de tiempo representa un paso de razonamiento, lo que permite al modelo planificar exhaustivamente antes de generar una frase. En comparación con los transformers tradicionales, nuestro enfoque ofrece dos ventajas clave: (1) al separar el pensamiento del habla, permite un razonamiento más estructurado y deliberado más allá de la generación token por token; y (2) al llevar a cabo todo el proceso de pensamiento en un espacio continuo, aumentamos significativamente el ancho de banda de información entre los pasos de razonamiento, lo que permite un pensamiento paralelo dentro de una sola pasada hacia adelante de MarCos.

Para pasar de un pensamiento a otro, la idea más sencilla es: Tener una correspondencia determinista entre el pensamiento actual y el siguiente. Sin embargo, esto podría ser subóptimo, ya que a menudo existen múltiples caminos lógicos posibles para un problema matemático, lo que hace que las correspondencias deterministas sean insuficientes para modelar la diversa distribución de los posibles pensamientos, limitando la exploración del modelo durante la inferencia.

Para abordar este problema, MarCos asigna cada pensamiento a una distribución multimodal de posibles pensamientos en el siguiente paso. Al hacerlo, podemos parametrizar explícitamente la aleatoriedad en el proceso de pensamiento, sin depender de la muestreo a nivel de token.

Primero presentaremos la estructura de MarCos en la Sección 3.1. Posteriormente, mostraremos la receta de entrenamiento de MarCos en la Sección 3.2. Específicamente, proponemos un esquema de entrenamiento en dos fases, inspirado en VAE (Kingma & Welling, 2013), para abordar la intratabilidad de las probabilidades de muestra debido al mapeo estocástico entre los pensamientos latentes.

3.1. Arquitectura del modelo

Esta sección explica cómo MarCos "piensa" y genera la solución a una pregunta. Como se muestra en la Figura 2, consta de tres capas separadas: comprensión, pensamiento y expresión, que proporcionan condiciones, probabilidades de transición y probabilidades de emisión para cHMM.

Comprensión. Dado la pregunta de entrada x de longitud n, utilizamos una capa de comprensión para convertirla en una secuencia de vectores de características: Hin = {h1, ..., hn}:

$$\mathrm{H}^{in} = \mathrm{Understanding}(x) \in \mathbb{R}^{n \times d}, \qquad (1)$$

donde d representa la dimensión latente.

Pensando. Como la variable oculta del cHMM, nuestro pensamiento continuo se manifiesta como el estado de un grupo de neuronas. Específicamente, inspirado en la evidencia neurocientífica sobre la especialización funcional en el cerebro humano (Kanwisher, 2010), utilizamos dos tipos de neuronas: Neudeep ∈RT ×d y Neushallow ∈RS×d, encargadas del pensamiento lento y rápido.

Además, Neudeep facilita un razonamiento más profundo, mientras que Neushallow representa un razonamiento más superficial y fácilmente verbalizable, ya que interactúa de manera más estrecha con la capa de habla (ver Ecuación 4). Aquí, T y S denotan el número de neuronas en cada grupo.

Luego, cada paso de pensamiento corresponde a una actualización de Neudeep y Neushallow dentro de una capa de pensamiento, basada en la característica de entrada Hin y sus propios estados previos.

Como se ha descrito anteriormente, una diferencia clave entre MarCos y los enfoques de razonamiento continuo existentes es que éste parametriza explícitamente la aleatoriedad del proceso de pensamiento a nivel de paso. Concretamente, introducimos una variable aleatoria auxiliar Rk ∈ Rτ×d para controlar el comportamiento de razonamiento en el paso k. τ representa aquí el factor de escalamiento de la longitud de Rk. En lugar de mapear directamente el pensamiento actual en el momento k-1 a la distribución multimodal y de alta dimensión de los pensamientos en el siguiente paso k, primero lo mapeamos a una distribución de Rk, que se encuentra en un espacio mejor definido y de menor dimensión. Luego, podemos mapear el pensamiento actual y la variable de control Rk de manera determinística al siguiente pensamiento. Para fomentar que cada dimensión de Rk represente un único factor aleatorio y estabilizar el proceso de entrenamiento, además añadimos una restricción de escasez a Rk, que se detallará en la Sección 3.2. Como se observa en las Secciones 4.1.2 y 4.2.3, la introducción de Rk facilita una aleatoriedad más controlable en el proceso de razonamiento. Por ejemplo, observamos que las diferentes dimensiones de Rk controlan eficazmente diferentes aspectos del pensamiento, incluyendo factores de alto nivel (p. ej., direcciones de búsqueda, profundidad del razonamiento) y elecciones de bajo nivel (p. ej., formato de expresión, longitud de la oración).

Específicamente, empleamos un predictor de aleatoriedad aprendible, g, para predecir la distribución de Rk, basándonos en la información de entrada Hin y en el pensamiento actual Neudeepk − 1 y Neushallowk − 1.

$$\boldsymbol{\mu}_k, \boldsymbol{\sigma}_k = g(\mathbf{Neu}_{k-1}^{deep}, \mathbf{Neu}_{k-1}^{shallow}, \mathrm{H}^{in}). \tag{2}$$

Luego, se extrae una muestra de Rk ~N(μk, σk) para guiar la transición del pensamiento actual al siguiente[1]:

$$\mathbf{Neu}_k^{deep}, \mathbf{Neu}_k^{shallow} = \text{Thinking}(\mathbf{Neu}_{k-1}^{deep}, \mathbf{Neu}_{k-1}^{shallow}, \mathrm{H}^{in}, \boldsymbol{R}_k). \tag{3}$$

La capa de procesamiento utiliza la atención bidireccional para facilitar este flujo de información y actualiza iterativamente las neuronas, estimulando un razonamiento paso a paso. Los valores iniciales de las neuronas, Neudeep0 y Neushallow0, son parámetros aprendibles.

Hablando. Después de cada paso de reflexión, algunos pensamientos en Neushallowk, como planes e ideas intermedias, pueden estar listos para ser expresados. Utilizamos un método de habla…

[1] La Eq. (3) puede extenderse para incluir todos los neuronas de pensamiento anteriores como entrada. En el Apéndice A, demostramos que esta configuración de primer orden no crea un cuello de botella de información en comparación con CoT.

layer to translate them into the discrete space of sentences:

$$\boldsymbol{s}_k = \text{Speaking}(\mathbf{Neu}_k^{shallow}). \tag{4}$$

A diferencia de los transformers tradicionales, en MarCos no depende del habla, por lo que nuestra etapa de habla es opcional para todos los pasos intermedios y puede omitirse, excepto en el paso final, donde solo se requiere una respuesta. Además, el uso del habla en cada paso es independiente de los demás, lo que los hace altamente paralelizable en comparación con los modelos tradicionales.

Además, dado que la aleatoriedad en MarCos se resuelve a nivel de paso en la etapa de pensamiento antes de que la capa de expresión genere oraciones específicas, se puede simular una aleatoriedad compleja “en un solo paso” en lugar de a través de múltiples “muestras iterativas”. Esto ofrece una nueva perspectiva para el desarrollo de modelos de lenguaje no-autoregresivos (NAR) con generación en un solo paso, y sugiere una vía prometedora para acelerar los modelos de lenguaje LLM basados en difusión (Gulrajani & Hashimoto, 2023; Sahoo et al., 2024). Para verificar esta idea, realizamos un experimento preliminar reemplazando la capa de expresión de MarCos con un decodificador NAR completamente. Como se muestra en la Sección 4.2.3, incluso la estrategia de decodificación NAR más simple aún obtiene un rendimiento competitivo, lo que proporciona una mayor aceleración para MarCos. Esto también destaca que nuestro modelo logra una clara separación entre el pensamiento y la expresión, asegurando que la calidad del razonamiento se mantenga estable mientras permite flexibilidad en la elección de la estrategia de expresión.

3.2. Entrenamiento

En esta sección, explicamos cómo entrenar a MarCos de manera efectiva. Asumiendo que tenemos una muestra de entrenamiento que consta de una pregunta x y su solución paso a paso y = {y1, ..., yN}. El objetivo del entrenamiento es encontrar los parámetros que maximicen la probabilidad de generar todos los yk. Sin embargo, dado que la muestreo ocurre en el proceso de pensamiento y no en el de habla, ya no tenemos una probabilidad condicional de forma cerrada para cada yk.

Para resolver esto, adaptamos el límite inferior de la evidencia (ELBO) utilizado en el entrenamiento de VAE para maximizar la probabilidad marginal de yk en cada paso. Específicamente, el ELBO general es:

$$\mathcal{L}_k^{\text{ELBO}} = \mathbb{E}_{q(\boldsymbol{R}_k|y_k)}\big[\log p(y_k|\boldsymbol{R}_k)\big] - \mathrm{KL}\big(q(\boldsymbol{R}_k|y_k) \,\|\, p(\boldsymbol{R}_k)\big), \tag{5}$$

que es una combinación de un término de reconstrucción y un término KL. En nuestro caso, el codificador VAE q se implementa mediante una función aprendible f (denominada “codificador de aleatoriedad” en la Figura 2), para mapear yk a la media y la varianza del posterior de Rk:

$$\hat{\boldsymbol{\mu}}_k, \hat{\boldsymbol{\sigma}}_k = f(y_k). \tag{6}$$

La decodificación de la VAE es simplemente un paso de razonamiento y expresión en las Ecuaciones 3 y 4, que mapea Rk de vuelta al espacio de las oraciones. Entonces, la pérdida de reconstrucción puede expandirse como:

$$\mathcal{L}_k^{re} = \mathbb{E}_{\boldsymbol{R}_k \sim \mathcal{N}(f(y_k))}\big[\log p(\boldsymbol{s}_k = y_k|\mathbf{Neu}_{k-1}^{deep}, \mathbf{Neu}_{k-1}^{shallow}, \mathrm{H}^{in}, \boldsymbol{R}_k)\big]. \tag{7}$$

A diferencia de los VAEs “vanilla” con priors gaussianos isotrópicos fijos, nuestro predictor de aleatoriedad, *g* en la Ecuación 2, proporciona un prior diferente para diferentes preguntas y pensamientos, tal como se describe en la Sección 3.1. Consecuentemente, nuestra divergencia KL entre el prior (Ecuación 2) y el posterior (Ecuación 6) es:

$$\mathcal{L}_k^{KL} = \mathrm{KL}\big(\mathcal{N}(f(y_k)) \,\|\, \mathcal{N}(g(\mathbf{Neu}_{k-1}^{deep}, \mathbf{Neu}_{k-1}^{shallow}, \mathrm{H}^{in}))\big). \tag{8}$$

Además, sin un control adecuado, el modelo podría codificar información semántica excesiva de yk en Rk a través del codificador aleatorio f. Esta información puede fusionarse fácilmente en Neushallowk en la Ecuación 3 para la reconstrucción de yk, creando un “atajo” de entrenamiento que impide que el modelo aprenda un razonamiento genuino. Para evitar esto, introducimos una pérdida de escasez adicional, Lsparsek = ‖Rk‖1. Esto obliga a Rk a almacenar solo la cantidad mínima de información que no puede ser inferida del contexto, al tiempo que proporciona un sesgo inductivo más fuerte para un razonamiento estructurado e interpretable. Por ejemplo, en cada paso de pensamiento, solo un pequeño conjunto de dimensiones necesita estar activo. Las dimensiones útiles para un razonamiento de amplitud no estarán activas durante un razonamiento de profundidad. Para reflejar este sesgo inductivo y fomentar que cada dimensión de Rk capture un "factor semántico único" en el razonamiento, nos inspiramos en autoencoders escasos e implementamos la función f como un transformador de d dimensiones seguido de una MLP que expande la dimensión de d a τ × d. La función de pérdida general de MarCos es:

$$\mathcal{L} = \sum_{k\in\{1,2,\ldots,N\}} \big[-\mathcal{L}_k^{re} + \mathcal{L}_k^{KL} + \lambda \mathcal{L}_k^{sparse}\big], \tag{9}$$

Donde λ es un hiperparámetro que controla la intensidad de la penalización de esparsidad. Para evitar la necesidad de seleccionar λ previamente, adoptamos una estrategia de ponderación dinámica que controla λ de forma adaptativa, buscando equilibrar la regularización de reconstrucción y esparsidad durante el entrenamiento (ver el Apéndice B). Además, para evitar el colapso del modelo debido a la optimización simultánea de la distribución posterior y la distribución previa, empleamos un esquema de entrenamiento en dos fases, donde primero optimizamos Lrek y Lsparsek con g fijo, y luego entrenamos g para minimizar LKLk con todos los demás parámetros fijados.

4. Experimentos

4.1. Evaluación en benchmarks sintéticos

Para comparar de manera sistemática las características fundamentales de MarCos y los transformers en la realización de razonamiento y modelado de la aleatoriedad, primero diseñamos tres tareas de razonamiento sintéticas, con ejemplos ilustrados en la Figura 3.

4.1.1. CONFIGURACIÓN EXPERIMENTAL

Tarea de nivel 1: Sumación Aleatoria (SA). Partiendo de una secuencia aleatoria de 2N números enteros no negativos de un dígito, en cada paso, se extrae un número entero no negativo de un dígito de forma uniforme y se añade a cada número de la secuencia.

Level-1 Task (Random Summation, $N = 5, K = 3$)
Query: **7 9 8 5 7 4 1 3 6 2**
Response: Step 1: (add 5) 12 14 13 10 12 9 6 8 11 7
Step 2: (add 2) 14 16 15 12 14 11 8 10 13 9
Step 3: 14+16=30

Level-2 Task (Random Pairing, $N = 5, K = 3$)
Query: **7 9 8 5 7 4 1 3 6 2**
Response: Step 1: {(7, 4), (9, 3), (7, 8), (5, 2), (1, 6)} 11 12 -1 7 15 3 7 6 -5 3
Step 2: {(12, 11), (-1, -5), (7, 7), (15, 3), (3, 6)} 1 23 -6 14 18 12 0 -3 4 9
Step 3: 1+23=24

Level-3 Task (Parallel 24)
Query: **6 9 10 11**
Response: Step 1: 9-6=3
Step 2: 10-9=1. 11/1=11
Step 3: 11+10=21. 21+9=30. 30-6=24

Figura 3. Descripción general de las tareas sintéticas. La oración de salida se muestra en negro, mientras que los segmentos grises se incluyen únicamente para facilitar la delimitación de los pasos y para comprender el proceso de razonamiento.

Tabla 1. Precisión en tareas sintéticas (N = 5, K = 3).

Model	RS	RP	P24
Transformer	0.999	0.589	0.170
MarCos	1.000	0.712	0.668

El proceso se repite por K pasos. En el último paso, se calcula la suma de los dos primeros números como resultado final. Esta tarea requiere las habilidades aritméticas más básicas y la modelización de una aleatoriedad muy simple en los sumandos.

Tarea de nivel 2: Emparejamiento Aleatorio (EA). Dada una secuencia aleatoria de 2N enteros de dígito único y no negativos, en cada paso, el modelo debe dividirla aleatoriamente en N pares. Para cada par (a, b), la primera posición se actualiza a (a + b) y la segunda posición a (a − b). Las demás configuraciones son las mismas que en la tarea de nivel 1 RS. En comparación con RS, esta tarea no solo implica cálculos más complejos, sino que también introduce una mayor complejidad en la aleatoriedad del emparejamiento. Debido a que se debe determinar una configuración de emparejamiento global antes de la generación de tokens, la aleatoriedad es más difícil de modelar mediante muestreo a nivel de token.

Tarea de nivel 3: Tarea 24 (T24). El clásico juego de 24 requiere que un modelo realice pruebas y errores hasta alcanzar el número 24. En T24, se pide a los modelos que consideren diferentes caminos en paralelo y que encuentren uno correcto en K pasos. Esta tarea exige un pensamiento paralelo y un mayor ancho de banda de información, ya que el modelo debe considerar la mayor cantidad de caminos de razonamiento posibles en cada paso. Simula cómo los humanos ajustan dinámicamente su estrategia de razonamiento durante la resolución de problemas.

4.1.2. RESULTADOS

La Tabla 1 muestra el rendimiento de MarCos y del transformer en las tres tareas. En la tarea RS, donde la aleatoriedad se puede absorber eficazmente en un proceso de generación secuencial de tokens, tanto MarCos como el transformer alcanzan un rendimiento perfecto. Sin embargo, el rendimiento del transformer se degrada significativamente en la tarea RP, que requiere modelar la aleatoriedad más estructurada de la partición aleatoria. Este resultado sugiere que la muestreo basado en tokens…

Tabla 2. Resultados de modelos entrenados desde cero. Todas las líneas de razonamiento continuo (0.5B) fueron entrenadas con datos de ecuaciones, siguiendo sus artículos originales. Se reportan los modelos continuos más destacados en **negrita**, y los segundos mejores en *cursiva*.

		GSM8K		SVAMP		MultiArith	
Model	Train (h)	Acc (%)	Test (s)	Acc (%)	Test (s)	Acc (%)	Test (s)
Training Data: Text							
CoT-SFT	$0.31_{\pm 0.01}$	$13.27_{\pm 0.59}$	$80.95_{\pm 5.78}$	$23.25_{\pm 0.39}$	$216.56_{\pm 11.88}$	$25.20_{\pm 0.47}$	$18.32_{\pm 1.81}$
MarCos	$1.13_{\pm 0.01}$	**17.97**$_{\pm 0.96}$	$17.53_{\pm 0.29}$	**24.39**$_{\pm 1.10}$	$34.14_{\pm 0.49}$	**23.67**$_{\pm 0.53}$	$5.90_{\pm 0.17}$
$-\ w/o\ \mathbf{Neu}^{deep}$	$1.12_{\pm 0.01}$	$16.91_{\pm 1.60}$	$17.28_{\pm 0.13}$	$22.04_{\pm 0.29}$	$34.83_{\pm 0.26}$	$20.28_{\pm 0.25}$	$5.68_{\pm 0.16}$
$-\ w/o\ \boldsymbol{R}_k$	$0.77_{\pm 0.01}$	$16.07_{\pm 0.00}$	$17.20_{\pm 0.10}$	$20.27_{\pm 0.00}$	$32.43_{\pm 0.11}$	$16.66_{\pm 0.00}$	$5.65_{\pm 0.12}$
Training Data: Equation							
CoT-SFT	$0.38_{\pm 0.01}$	$20.31_{\pm 0.85}$	$32.97_{\pm 1.03}$	$27.92_{\pm 0.36}$	$162.12_{\pm 7.43}$	$41.48_{\pm 0.50}$	$9.61_{\pm 0.91}$
iCoT-SI	$0.43_{\pm 0.03}$	$14.81_{\pm 0.55}$	$13.11_{\pm 1.62}$	$18.46_{\pm 0.54}$	$17.30_{\pm 0.31}$	$21.33_{\pm 0.43}$	$2.32_{\pm 0.05}$
Coconut	$1.50_{\pm 0.25}$	$15.27_{\pm 0.05}$	$59.53_{\pm 0.53}$	$17.76_{\pm 0.01}$	$106.74_{\pm 1.46}$	$15.50_{\pm 0.01}$	$15.98_{\pm 0.28}$
CoLaR	$1.17_{\pm 0.00}$	$15.45_{\pm 0.42}$	$17.20_{\pm 0.45}$	$23.09_{\pm 0.09}$	$35.80_{\pm 0.45}$	$38.60_{\pm 0.55}$	$3.80_{\pm 0.45}$
CODI	$0.68_{\pm 0.09}$	$\underline{16.07}_{\pm 0.17}$	$8.58_{\pm 0.25}$	$\underline{26.24}_{\pm 0.22}$	$24.88_{\pm 0.26}$	$\underline{40.03}_{\pm 0.25}$	$3.48_{\pm 0.24}$
MarCos	$1.03_{\pm 0.01}$	**24.11**$_{\pm 0.97}$	$17.76_{\pm 0.04}$	**27.77**$_{\pm 0.32}$	$34.02_{\pm 0.11}$	**42.33**$_{\pm 0.56}$	$6.02_{\pm 0.28}$
$-\ w/o\ \mathbf{Neu}^{deep}$	$0.99_{\pm 0.01}$	$21.07_{\pm 0.76}$	$17.09_{\pm 0.30}$	$25.83_{\pm 0.19}$	$35.24_{\pm 0.11}$	$38.38_{\pm 1.04}$	$6.00_{\pm 0.29}$
$-\ w/o\ \boldsymbol{R}_k$	$0.74_{\pm 0.01}$	$23.99_{\pm 0.00}$	$17.67_{\pm 0.09}$	$26.90_{\pm 0.00}$	$33.78_{\pm 0.54}$	$42.17_{\pm 0.00}$	$5.99_{\pm 0.08}$

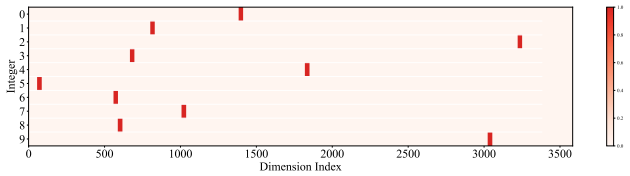


Figura 4. Codificación de un-a-uno emergente de aleatoriedad en Rk. Cada entero elegido al azar activa consistentemente una dimensión única en Rk, independientemente de la secuencia de entrada.

Es más adecuado para la aleatoriedad que admite una factorización secuencial, pero se ve severamente limitado al tratar con la aleatoriedad estructurada que implica un pensamiento global.

En la tarea P24 más desafiante, MarCos alcanza una mejora considerable en comparación con el transformer. Esta gran diferencia destaca la ventaja de la capacidad de procesamiento continuo de alta anchura en nuestra capa de razonamiento. Al mantener implícitamente todas las rutas de razonamiento paralelas, MarCos explora el espacio de soluciones de manera mucho más efectiva que el transformer, lo cual es importante en entornos con ramificaciones complejas y replanteamientos frecuentes.

Un fenómeno interesante es que nuestro modelo aprende a almacenar una aleatoriedad perfectamente limpia e independiente del contexto en la variable aleatoria Rk. Por ejemplo, en la tarea RS, cada entero elegido aleatoriamente activa consistentemente una dimensión única de Rk, mientras que todas las demás dimensiones permanecen inactivas (Figura 4). Este patrón se mantiene en TODOS los inputs. Además, durante la inferencia, si fijamos Neudeepk − 1 y Neushallowk − 1 y modificamos manualmente Rk en un paso k dado, la decodificación de sk y todos los pasos de razonamiento posteriores cambian consistentemente de acuerdo con esta modificación, sin errores (precisión = 100%). Estos hallazgos demuestran que MarCos separa explícitamente la aleatoriedad del procesamiento contextual de manera estructurada e interpretable. Las neuronas de pensamiento son responsables del procesamiento de la información contextual y realizando los cálculos de razonamiento principales, mientras que Rk controla la dirección del razonamiento (es decir, la aleatoriedad).

4.2. Evaluación en benchmarks del mundo real

Tras validar la eficacia de nuestro MarCos en tareas sintéticas, ahora nos enfocamos en tareas del mundo real a mayor escala, con el fin de evaluar aún más su potencial en problemas más complejos.

4.2.1. CONFIGURACIÓN EXPERIMENTAL

Conjuntos de datos y tareas. Siguiendo (Tan et al., 2025), entrenamos nuestro modelo en el conjunto de datos GSM8K-Aug (Deng et al., 2023), que contiene 385.000 muestras de entrenamiento. Posteriormente, evaluamos el modelo en el conjunto de pruebas original de GSM8K (tarea específica), así como en dos conjuntos de referencia fuera del dominio: (1) SVAMP (Patel et al., 2021), que consta de 4.138 problemas aritméticos construidos a través de sutiles variaciones en la redacción y perturbaciones semánticas, con el objetivo de evaluar la robustez del razonamiento; (2) MultiArith (Roy & Roth, 2015), una colección de 600 problemas extraídos de MAWPS (Koncel-Kedziorski et al., 2016) que requieren razonamiento de múltiples pasos. Nuestras métricas de evaluación incluyen (1) Precisión (Acc), que refleja la capacidad de razonamiento; (2) Tiempo de Entrenamiento (Train), que mide el costo de tiempo para el entrenamiento del modelo; y (3) Tiempo de Prueba (Test), que mide la eficiencia de la inferencia contando el tiempo total requerido para generar soluciones para todos los ejemplos de prueba. Se proporcionan descripciones más detalladas de la configuración de evaluación en el Apéndice C.

Baselines. Nos comparamos con (1) CoT (Wei et al., 2022), que entrena modelos basados en transformadores con pares problema-solución y rutas de razonamiento explícitas, (2) iCoT-SI (Deng et al., 2024), que gradualmente elimina los pasos de razonamiento explícitos durante el entrenamiento para fomentar el razonamiento implícito, (3) Co-

Tabla 3. Resultados de modelos inicializados a partir de puntos de control preentrenados. Para MarCos, eliminamos Rk ya que los pesos preentrenados no son adecuados para su función estocástica de forma natural.

Model	GSM8K	SVAMP	MultiArith
Backbone: Qwen2.5-0.5B			
CoT	41.6	71.9	88.3
Coconut	28.6	60.7	88.0
CoLaR	36.0	61.7	82.2
CODI	40.8	71.2	87.6
MarCos(w/o $\boldsymbol{R}_k$)	**43.8**	**75.1**	**98.3**
Backbone: LLaMA-3.2-1B-Instruct			
CoT	**61.6**	66.7	**99.3**
Coconut	45.3	48.8	90.1
CoLaR	40.1	54.9	96.1
CODI	55.6	61.1	91.3
MarCos(w/o $\boldsymbol{R}_k$)	56.5	**68.5**	97.8

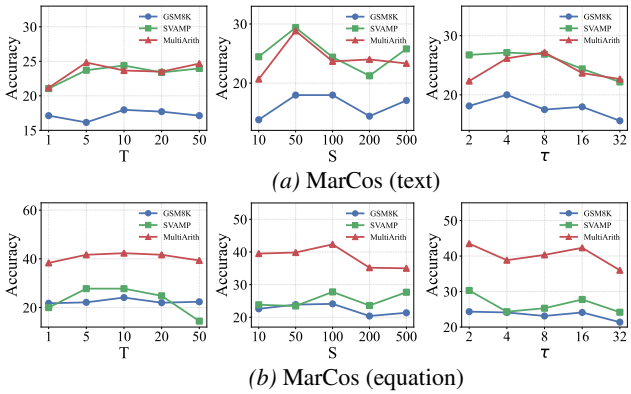
conut (Hao et al., 2024), que trata las representaciones latentes como un token especial e las introduce junto con los tokens de palabras en un modelo autoregresivo; (4) CoLaR (Tan et al., 2025), que comprime los tokens de razonamiento y aprende a predecir la distribución de estas representaciones comprimidas; y (5) CODI (Shen et al., 2025), que se auto-refina las activaciones ocultas en todas las capas utilizando el token de destilación seleccionado.

Detalles de implementación. Implementamos las capas de comprensión, pensamiento, habla y el codificador de aleatoriedad como modelos de 0.5B (estructura detallada en el Apéndice C). El predictor de aleatoriedad es una red MLP de dos capas. Para una comparación justa, implementamos modelos de referencia con parámetros de 0.5B y 3B. Para nuestro modelo y CoT-SFT, incluimos una versión basada en ecuaciones para una comparación directa y una versión basada en texto para aplicaciones del mundo real. Para otros modelos de referencia, seguimos sus recetas originales, entrenando primero un CoT explícito para la inicialización y luego afinando únicamente con ecuaciones estructuradas.

Dado que MarCos representa una arquitectura de razonamiento fundamentalmente nueva, evaluamos principalmente su eficacia entrenándolo desde cero para asegurar que las capacidades de razonamiento observadas provengan directamente de la arquitectura en sí, y no de conocimientos externos obtenidos a través del preentrenamiento. Para una comparación justa, todas las líneas de base utilizadas en los experimentos principales se entrenan desde cero utilizando los mismos datos. Se proporcionan más detalles sobre la configuración experimental en el Apéndice C. Además, discutimos brevemente el preentrenamiento de MarCos en el Apéndice G, que dejamos para trabajos futuros.

4.2.2. RESULTADOS PRINCIPALES

Como se muestra en las Tablas 2 y 5, MarCos supera a los modelos de referencia existentes en términos de precisión, al tiempo que ofrece un aumento de hasta 15.7 veces en velocidad en comparación con los CoTs basados en transformadores tradicionales. Específicamente, MarCos (ecuación) alcanza una precisión del 8.04%.



(a) MarCos (text)

(b) MarCos (equation)

Figura 5. Rendimiento de MarCos con diferentes números de neuronas y variables aleatorias.

Mejora con respecto a la línea de base CODI más sólida en GSM8K. Además de su rendimiento dentro del dominio, MarCos obtiene mejoras de hasta el 2.30% en SVAMP y MultiArith, lo que demuestra una fuerte generalización fuera del dominio. Además, MarCos supera a CoT-SFT en la mayoría de los escenarios. En particular, cuando se entrena con supervisión textual, MarCos mejora la precisión en un 4.70% en GSM8K. Estos resultados resaltan su capacidad para ejecutar razonamiento implícito y profundo de manera efectiva.

En términos de eficiencia, MarCos no solo supera a los CoTs basados en tokens, sino que también alcanza tiempos de inferencia más rápidos que muchos de los modelos de referencia de 0.5B. Esto demuestra que modelar la razón a nivel de pensamiento y separar el pensamiento y el habla, ofrecen ventajas computacionales significativas. De manera notable, MarCos requiere menos tiempo de entrenamiento que todos los modelos de 3B, e incluso algunos modelos de 0.5B. Estos resultados demuestran que nuestro esquema de entrenamiento en dos etapas es tanto efectivo como excepcionalmente eficiente.

Estudio de Ablación. Para demostrar la importancia de los diferentes componentes de MarCos, lo comparamos con dos variantes: una sin Neudeep, que elimina las neuronas de pensamiento profundo; y otra sin Rk, que elimina el controlador de aleatoriedad.

En la Tabla 2, observamos que eliminar cualquiera de estos componentes provoca una disminución significativa en el rendimiento. También descubrimos que la importancia relativa de Neudeep y Rk depende del tipo de supervisión. Cuando se entrenan con ecuaciones, eliminar Rk tiene un impacto menor que Neudeep, mientras que con supervisión textual ocurre lo contrario. Esto se debe a que las ecuaciones presentan muy poco azar inherente, mientras que los textos pueden exhibir una mayor variabilidad en aspectos como la longitud y el formato, lo cual será validado en la Sección 4.2.3. Proporcionamos estudios adicionales de ablación, incluyendo la dirección de atención, la estrategia de inicialización y el λ fijo, en el Apéndice D.

Intentando la Inicialización con Transformadores Preentrenados. En la Tabla 3, intentamos inicializar todas las capas de MarCos utilizando los pesos preentrenados de Qwen2.5-0.5B y LLaMA-3.2-1B-Instruct, y también inicializamos todas las líneas de base de manera correspondiente, para lograr una comparación justa. Sin embargo, observamos que los pesos de los transformadores preentrenados no son naturalmente adecuados para la aleatoriedad.

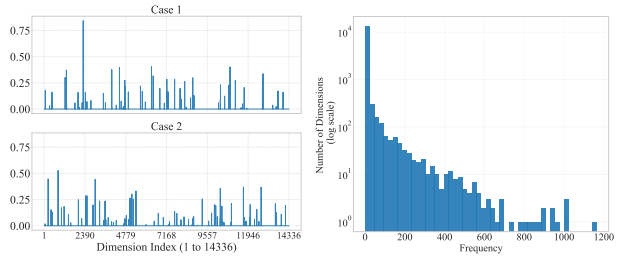


Figura 6. Izquierda: Rk para tres casos en el paso 1; Derecha: Distribución de frecuencia de todas las dimensiones en Rk, factor, ya que desempeña un papel fundamentalmente diferente al de los tokens estándar. Por lo tanto, eliminamos Rk en este escenario para asegurar una comparación clara. En este escenario, MarCos continúa obteniendo un rendimiento superior en todos los tres benchmarks. Particularmente, obtiene ganancias significativas sobre CoT en SVAMP (+3.2% para Qwen; +1.8% para LLaMA) y MultiArith (+10.0% para Qwen). Estos resultados reflejan que nuestra arquitectura puede inicializar componentes parciales a partir de puntos de control existentes, aprovechando su conocimiento pre-entrenado.

4.2.3. ANÁLISIS Y DISCUSIÓN

¿Más neuronas para el pensamiento, mejor rendimiento? Inicialmente, investigamos cómo la escala de las neuronas de pensamiento y las variables aleatorias influyen en el rendimiento del modelo, variando el número de neuronas de pensamiento (T ∈{1, 5, 10, 20, 50}), neuronas de pensamiento superficiales (S ∈{10, 50, 100, 200, 500}), y las variables aleatorias (τ ∈{2, 4, 8, 16, 32}) como se muestra en la Figura 5.

Para ambos tipos de datos, a medida que aumentan el número de neuronas T y S, el rendimiento del modelo muestra un aumento inicial, seguido de un período de estabilización o disminución. Esto sugiere que el rendimiento está inicialmente limitado por la capacidad de procesamiento del modelo, pero más allá de un cierto punto, añadir más neuronas podría introducir ruido o conducir a un sobreajuste.

En relación con la escala de aleatoriedad τ, observamos fenómenos diferentes en los modelos basados en texto y en ecuaciones. Para la configuración de ecuaciones, un τ grande generalmente produce un rendimiento peor, lo cual es consistente con nuestra hipótesis anterior en la Sección 4.2.2, que sugiere que la supervisión de ecuaciones contiene relativamente poca variabilidad. En contraste, para la configuración basada en texto, se obtiene un buen rendimiento únicamente con un τ grande, lo que indica que se requiere un mayor grado de estocasticidad para capturar adecuadamente las diversas variaciones del lenguaje natural.

¿Cómo controla Rk la aleatoriedad del razonamiento? Además de analizar la variable aleatoria Rk en tareas sintéticas descritas en la Sección 4.1.2, ampliamos nuestro estudio a tareas de razonamiento en el mundo real. Como se muestra en la Figura 6, diferentes datos activan diferentes dimensiones de Rk. Para cuantificar este efecto, definimos dimensiones con valores superiores a 0.1 como activadas y contamos la frecuencia de activación en todas las muestras. La parte derecha de la Figura 6 muestra un patrón de cola larga: mientras que la mayoría de las dimensiones se activan solo una vez, un pequeño subconjunto de dimensiones se activan con una frecuencia muy alta. Esto sugiere que…

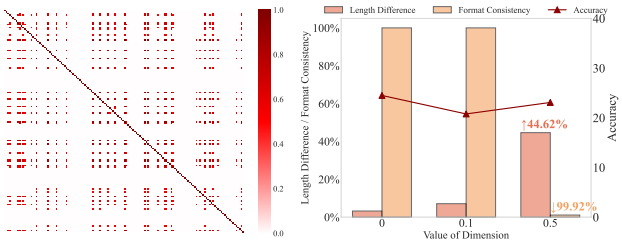


Figura 7. Izquierda: Mapa de correlación de las dimensiones más activadas (el 1% más frecuente) en Rk; Derecha: Resultados de la intervención de un grupo de dimensiones altamente correlacionadas en Rk.

La información aleatoria puede estar controlada por las dimensiones de alta frecuencia, mientras que la aleatoriedad específica del caso se refleja en las de baja frecuencia.

Para verificar esto, seleccionamos las 1% dimensiones que se activan con mayor frecuencia y calculamos sus coeficientes de correlación de Pearson en la Figura 7. De manera interesante, observamos que varias dimensiones están fuertemente correlacionadas entre sí. En el Apéndice E, visualizamos aún más esta estructura de correlación como un grafo, donde identificamos un grupo de cuatro dimensiones que podrían representar una forma de "control combinatorio". Específicamente, identificamos que estas dimensiones controlan principalmente dos factores: (1) la longitud de la salida, es decir, el número de tokens en los pasos de razonamiento, y (2) el formato de la salida, es decir, si la respuesta final está precedida por el marcador "####". Para ilustrarlo, intervenimos manualmente en estas dimensiones estableciendo sus valores a {0, 0.1, 0.5} para todas las muestras de datos. La Figura 7 muestra que con valores más altos, las soluciones generadas se vuelven considerablemente más largas. Por ejemplo, a 0.5, la longitud promedio aumenta en un 44.62%. Al mismo tiempo, la rigidez del formato disminuye: a 0 y 0.1, casi todas las respuestas conservan el prefijo "####", mientras que a 0.5 la proporción disminuye drásticamente a 1.08%. Crucialmente, a pesar de estos cambios, la precisión del razonamiento se mantiene en gran medida sin afectar (línea roja). Estos hallazgos indican que estas dimensiones controlan de forma independiente factores de aleatoriedad como la verbosidad y el formato, sin codificar el contenido esencial del razonamiento.

Para la información específica de cada caso, realizamos un análisis caso por caso en el Apéndice F. Inicialmente, observamos que varias dimensiones influyen en la profundidad del razonamiento. Por ejemplo, cuando se establecen las dimensiones A o C en 0.5, ambos casos muestran un razonamiento más profundo en el Paso 1, completando tanto el Paso 1 como el Paso 2 simultáneamente. En segundo lugar, identificamos roles funcionales distintos para ciertas dimensiones. En el Caso 1, las dimensiones B controlan si las unidades se expresan explícitamente en las ecuaciones (por ejemplo, "4" vs. "4 bolígrafos"), mientras que en el Caso 2, las dimensiones D determinan si los números se representan como fracciones ("20/100") o como decimales ("0.20"). Estos hallazgos reflejan que nuestro módulo variable captura de manera efectiva las variaciones sutiles en el razonamiento.

Potencial para la decodificación no autoregresiva en la etapa de habla. Una de las principales ventajas de MarCos es la separación explícita…

Comparación entre el pensamiento y el habla. Esto implica que una idea clara sobre lo que se va a decir ya está formada antes de comenzar la fase de habla. Como resultado, habrá menos aleatoriedad y menor correlación entre los tokens durante la decodificación de la oración, lo que hace que MarCos sea especialmente adecuado para la generación no-autoregresiva más rápida. Verificamos esta idea entrenando directamente la capa de habla para generar 128 tokens en una sola pasada. En la Tabla 7, incluso con esta receta de entrenamiento sencilla, MarCos aún logra un rendimiento comparable al de modelos de referencia como iCoT-SI (ver Tabla 2). Este hallazgo destaca el potencial de MarCos para integrar estrategias NAR más avanzadas, lo que representa una importante dirección futura.

5. Conclusión

En este trabajo, presentamos MarCos, una mejora de la arquitectura transformer que realiza el razonamiento paso a paso completamente en un espacio continuo, modelado como una cadena de Markov oculta. A través de extensas pruebas, demostramos que MarCos no solo supera a los transformers tradicionales, sino que también supera a los métodos de razonamiento continuo existentes, al tiempo que ofrece una mejora significativa en la velocidad de inferencia. En el futuro, nuestra principal prioridad es verificar la efectividad de MarCos cuando se preentrena con corpus masivos y desarrollar algoritmos de aprendizaje por refuerzo específicos para el entrenamiento posterior de MarCos. Para más discusiones sobre las limitaciones y las futuras direcciones de este trabajo, por favor, consulte el Apéndice G.

Declaración de impacto

Este artículo presenta un trabajo cuyo objetivo es avanzar en el campo del aprendizaje automático. Existen numerosas posibles consecuencias para la sociedad derivadas de nuestro trabajo, ninguna de las cuales sentimos que requiera ser resaltada específicamente.

Referencias

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Deng, Y., Prasad, K., Fernandez, R., Smolensky, P., Chaudhary, V., and Shieber, S. Implicit chain of thought reasoning via knowledge distillation. *arXiv preprint arXiv:2311.01460*, 2023.

Deng, Y., Choi, Y., and Shieber, S. From explicit cot to implicit cot: Learning to internalize cot step by step. *arXiv preprint arXiv:2405.14838*, 2024.

Fedorenko, E. and Varley, R. Language and thought are not the same thing: evidence from neuroimaging and neurological patients. *Annals of the New York Academy of Sciences*, 1369(1):132–153, 2016.

Fedorenko, E., Piantadosi, S. T., and Gibson, E. A. Language is primarily a tool for communication rather than thought. *Nature*, 630(8017):575–586, 2024.

Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A., and Goldstein, T. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.

Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D., and Papailiopoulos, D. Looped transformers as programmable computers. In *International Conference on Machine Learning*, pp. 11398–11442. PMLR, 2023.

Gulrajani, I. and Hashimoto, T. B. Likelihood-based diffusion language models. *Advances in Neural Information Processing Systems*, 36:16693–16715, 2023.

Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J., and Tian, Y. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.

Kambhampati, S., Valmeekam, K., Guan, L., Verma, M., Stechly, K., Bhambri, S., Saldyt, L. P., and Murthy, A. B. Position: Llms can't plan, but can help planning in llm-modulo frameworks. In *Forty-first International Conference on Machine Learning*, 2024.

Kanwisher, N. Functional specificity in the human brain: a window into the functional architecture of the mind. *Proceedings of the national academy of sciences*, 107 (25):11163–11170, 2010.

Kingma, D. P. and Welling, M. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

Koncel-Kedziorski, R., Roy, S., Amini, A., Kushman, N., and Hajishirzi, H. Mawps: A math word problem repository. In *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies*, pp. 1152–1157, 2016.

Lioubashevski, D., Schlank, T., Stanovsky, G., and Goldstein, A. Looking beyond the top-1: Transformers determine top tokens in order. *arXiv preprint arXiv:2410.20210*, 2024.

Miao, N., Rainforth, T., Mathieu, E., Dubois, Y., Teh, Y. W., Foster, A., and Kim, H. Learning instance-specific augmentations by capturing local invariances. In *International Conference on Machine Learning*, pp. 24720–24736. PMLR, 2023.

Patel, A., Bhattamishra, S., and Goyal, N. Are nlp models really able to solve simple math word problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2080–2094, 2021.

Roy, S. and Roth, D. Solving general arithmetic word problems. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 1743–1752, 2015.

Sahoo, S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J., Rush, A., and Kuleshov, V. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.

Shen, Z., Yan, H., Zhang, L., Hu, Z., Du, Y., and He, Y. Codi: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*, 2025.

Sprague, Z. R., Yin, F., Rodriguez, J. D., Jiang, D., Wadhwa, M., Singhal, P., Zhao, X., Ye, X., Mahowald, K., and Durrett, G. To cot or not to cot? chain-of-thought helps mainly on math and symbolic reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025.

Tan, W., Li, J., Ju, J., Luo, Z., Luan, J., and Song, R. Think silently, think fast: Dynamic latent compression of llm reasoning chains. *arXiv preprint arXiv:2505.16552*, 2025.

Valmeekam, K., Marquez, M., Sreedharan, S., and Kambhampati, S. On the planning abilities of large language models-a critical investigation. *Advances in Neural Information Processing Systems*, 36:75993–76005, 2023.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Wei, X., Liu, X., Zang, Y., Dong, X., Cao, Y., Wang, J., Qiu, X., and Lin, D. Sim-cot: Supervised implicit chain-of-thought. *arXiv preprint arXiv:2509.20317*, 2025.

Yang, L., Lee, K., Nowak, R. D., and Papailiopoulos, D. Looped transformers are better at learning learning algorithms. In *The Twelfth International Conference on Learning Representations*, 2024.

Zhang, C. et al. Cot2: Scaling reasoning via continuous thought augmentation. *arXiv preprint*, 2025.

Zhu, R.-J., Wang, Z., Hua, K., Zhang, T., Li, Z., Que, H., Wei, B., Wen, Z., Yin, F., Xing, H., et al. Escalar los factores latentes reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025.

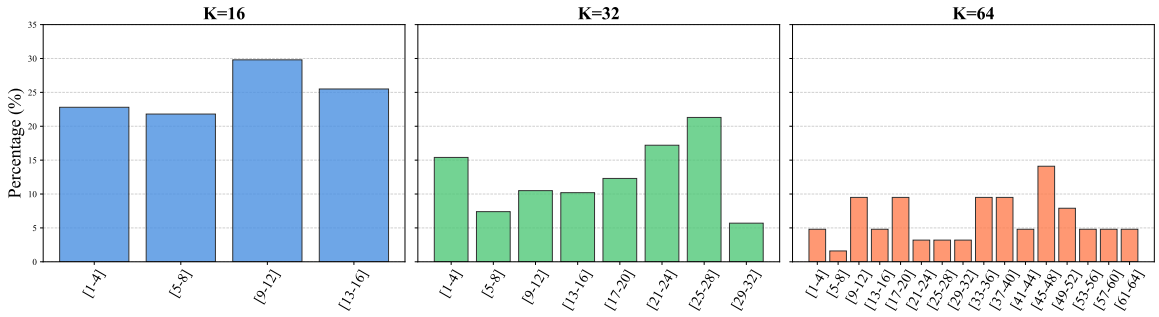


Figura 8. Distribución de los pasos de origen seleccionados en el momento de la prueba para la tarea de dependencia a largo plazo (%). Cada intervalo cubre 4 pasos anteriores.

A. Asunción de Markov y Dependencias a Largo Plazo

Una preocupación natural es si la suposición de Markov de primer orden en MarCos limita su capacidad para retener información a lo largo de múltiples pasos de razonamiento. Queremos destacar que el pensamiento en cada paso se representa mediante vectores continuos de alta dimensión, que se actualizan sin discretización ni "información bottleneck", lo que permite que el modelo tenga la capacidad inductiva de codificar toda la información previa necesaria. Para validar esto empíricamente, diseñamos una tarea en la que el modelo genera aleatoriamente 5 números durante K-1 pasos y debe copiar aleatoriamente los números de uno de estos pasos previos en el paso K, lo que requiere que el modelo retenga toda la información de los K-1 pasos previos. Como se muestra en la Tabla 4, MarCos aprende con éxito las dependencias a largo plazo en K = 16 (88.2%) y mantiene un rendimiento no trivial incluso en K = 32 y K = 64, donde CoT colapsa por completo.

Tabla 4. Prueba de dependencia a largo plazo: precisión en la retención de información de todos los pasos anteriores.

K	16	32	64
CoT	92.5	0.0	0.1
MarCos	88.2	8.8	3.2

Para abordar la preocupación de que el modelo podría estar “tramando” al copiar siempre de los pasos más cercanos, analizamos la distribución de los pasos seleccionados por el modelo durante la fase de prueba. El paso de copia se selecciona aleatoriamente durante el entrenamiento, por lo que el modelo no tiene incentivo para degenerar en una estrategia de selección sesgada. La Figura 8 muestra la distribución de selección durante la prueba. Con K = 16, las selecciones del modelo están bien distribuidas en todos los contenedores de pasos anteriores. Con valores de K más grandes, aunque la distribución se vuelve ligeramente más desigual (como se esperaría en tareas más difíciles), el modelo sigue cubriendo todas las posiciones de pasos, en lugar de concentrarse en un subconjunto fijo, lo que confirma la retención genuina de información a largo plazo.

B. Estrategia de ponderación dinámica.

Dado que es difícil predefinir un equilibrio adecuado entre la pérdida de reconstrucción y la regularización de esparcimiento, adoptamos una estrategia de ponderación dinámica sencilla (Miao et al., 2023) que controla directamente el nivel de esparcimiento esperado durante el entrenamiento.

Específicamente, introducimos una densidad de objetivo β y ajustamos λ en función de la densidad Sbatch del lote de entrenamiento actual t:

$$\lambda_{t+1} = \begin{cases} 1.01 \cdot \lambda_t, & \text{if } S_{batch} > \beta, \\ 0.99 \cdot \lambda_t, & \text{if } S_{batch} \leq \beta. \end{cases} \tag{10}$$

Inicializamos λ0 a $10-4$ para evitar una regularización excesivamente fuerte en las primeras etapas de entrenamiento. Empíricamente, observamos que esta estrategia mantiene de manera constante el nivel de esparcimiento en un valor objetivo β, evitando una optimización excesivamente agresiva del esparcimiento, lo que de otro modo podría provocar que la pérdida de reconstrucción caiga prematuramente en mínimos locales deficientes.

C. Detalles de Implementación y Evaluación

En MarCOS, la capa de comprensión, la capa de habla y el codificador de aleatoriedad se implementan mediante la superposición de capas de transformador estándar, utilizando la misma configuración de modelo que Qwen2.5-0.5B. En contraste, la capa de "pensamiento" es un módulo especializado que actualiza un conjunto de neuronas y variables aleatorias. Por lo tanto, es una pila de capas de transformador con atención bidireccional.

Tabla 5. Resultados de modelos entrenados desde cero. Todas las líneas de razonamiento continuo (3B) fueron entrenadas con datos de ecuaciones, siguiendo sus artículos originales. Se reportan los modelos continuos más destacados en **negrita**, y los segundos mejores en *cursiva*.

		GSM8K		SVAMP		MultiArith	
Model	Train (h)	Acc (%)	Test (s)	Acc (%)	Test (s)	Acc (%)	Test (s)
Training Data: Text							
CoT-SFT	$1.81_{\pm 0.00}$	$12.66_{\pm 0.20}$	$177.37_{\pm 1.91}$	$24.24_{\pm 0.50}$	$537.78_{\pm 12.14}$	$23.44_{\pm 1.40}$	$58.20_{\pm 2.50}$
MarCos	$1.13_{\pm 0.01}$	**17.97**$_{\pm 0.96}$	$17.53_{\pm 0.29}$	**24.39**$_{\pm 1.10}$	$34.14_{\pm 0.49}$	**23.67**$_{\pm 0.53}$	$5.90_{\pm 0.17}$
Training Data: Equation							
CoT-SFT	$1.23_{\pm 0.00}$	$22.70_{\pm 0.36}$	$54.07_{\pm 2.16}$	$27.25_{\pm 0.46}$	$125.47_{\pm 7.67}$	$50.20_{\pm 1.60}$	$10.73_{\pm 0.17}$
iCoT-SI	$2.40_{\pm 0.10}$	$14.08_{\pm 0.67}$	$13.23_{\pm 0.06}$	$16.26_{\pm 0.27}$	$29.02_{\pm 0.44}$	$19.67_{\pm 0.58}$	$3.49_{\pm 0.05}$
Coconut	$12.62_{\pm 1.06}$	$9.95_{\pm 0.04}$	$127.35_{\pm 0.44}$	$13.53_{\pm 0.11}$	$735.14_{\pm 3.80}$	$8.78_{\pm 0.19}$	$26.56_{\pm 0.66}$
CoLaR	$2.21_{\pm 0.00}$	$\underline{22.60}_{\pm 0.19}$	$38.13_{\pm 0.21}$	$\underline{25.83}_{\pm 0.22}$	$129.73_{\pm 0.96}$	$\underline{41.83}_{\pm 0.44}$	$18.93_{\pm 0.73}$
MarCos	$1.03_{\pm 0.01}$	**24.11**$_{\pm 0.97}$	$17.76_{\pm 0.04}$	**27.77**$_{\pm 0.32}$	$34.02_{\pm 0.11}$	**42.33**$_{\pm 0.56}$	$6.02_{\pm 0.28}$

permitir que cada neurona considere el estado de todas las demás. En otras palabras, no modificamos los cálculos internos de los transformers, ni es el objetivo de nuestro trabajo. En cambio, reutilizamos el transformer estándar para lograr roles fundamentalmente diferentes en MarCos. En particular, a través del diseño de la capa de razonamiento, demostramos que una estructura similar a un transformer puede realizar un razonamiento de un solo paso en una sola pasada.

Las cantidades de neuronas profundas y superficiales están configuradas en $T = 10$, $S = 100$. El factor de escala de la variable de aleatoriedad Rk se establece en $\tau = 16$. MarCos se entrena con AdamW, utilizando una tasa de aprendizaje de $1e-4$ y un decaimiento de peso de 0.01. El tamaño del lote de entrenamiento es 256, y ambas fases de entrenamiento se ejecutan durante 10 épocas. Para simplificar la prototipación, el número de pasos de pensamiento se fija en $K = 3$, aunque nuestro modelo también admite decidir dinámicamente este número. Para los datos de texto, cada punto (".") se considera un paso separado. Si una muestra contiene más de tres pasos, los pasos originales se agrupan y recombinan en tres pasos. Para una comparación justa, el modelo de referencia CoT-SFT se entrena durante 20 épocas. Todos los experimentos se llevan a cabo en un servidor con 8 GPUs H200.

En nuestra evaluación, el Tiempo de Entrenamiento se define como el tiempo real transcurrido para completar una época de entrenamiento. Para asegurar una comparación justa, para los modelos de referencia que requieren una etapa de pre-entrenamiento con un modelo CoT explícito, el tiempo de entrenamiento reportado es la suma de (i) el tiempo necesario para entrenar el modelo CoT explícito durante una época, y (ii) el tiempo necesario para entrenar su propio modelo durante una época. Para nuestro modelo, el tiempo de entrenamiento reportado es la suma de las dos fases de entrenamiento, cada una medida durante una época. El Tiempo de Prueba es el tiempo total requerido para inferir todos los ejemplos de prueba en una sola GPU, utilizando un tamaño de lote de 64. Sin embargo, la implementación original de Coconut solo admite la inferencia con un tamaño de lote $= 1$. Intentamos modificarlo a 64, pero no observamos una mejora significativa en la eficiencia. Por lo tanto, para Coconut reportamos el tiempo de su configuración predeterminada, es decir, 4 GPUs con un tamaño de lote $= 1$.

D. Estudios adicionales de ablación

Proporcionamos resultados adicionales para examinar la sensibilidad de MarCos a las elecciones arquitectónicas y los hiperparámetros. (1) Dirección de atención. Comparamos la atención bidireccional (nuestra opción predeterminada) con la atención unidireccional en la capa de razonamiento. Como se muestra en la Tabla 6, la atención bidireccional supera consistentemente a la atención unidireccional, lo que confirma que permitir que las neuronas presten atención a todas las demás es beneficioso para el razonamiento. (2) Estrategia de inicialización. Las diferencias de rendimiento entre nuestra inicialización predeterminada con Xavier y la inicialización Normal para la capa de razonamiento son mínimas en todos los tres benchmarks, lo que indica que MarCos es robusto a la elección de la inicialización. (3) λ fijo. Aunque MarCos adopta una estrategia de ponderación dinámica para λ, también evaluamos valores fijos para obtener una evaluación completa. Según la Tabla 6, fijar λ a cualquiera de los valores probados degrada el rendimiento en comparación con la estrategia dinámica, lo que motiva la necesidad de un equilibrio dinámico.

Tabla 6. Eliminación de la dirección de la atención y la estrategia de inicialización (datos de ecuaciones).

Ablation	Setting	GSM8K	SVAMP	MultiArith
Attention	Bidirectional (default)	24.11	27.77	42.33
Attention	Unidirectional	22.74	25.35	38.82
λ	10^{-4}	21.32	23.98	40.54
λ	5×10^{-4}	20.83	23.90	39.43
λ	10^{-3}	20.83	23.82	36.39

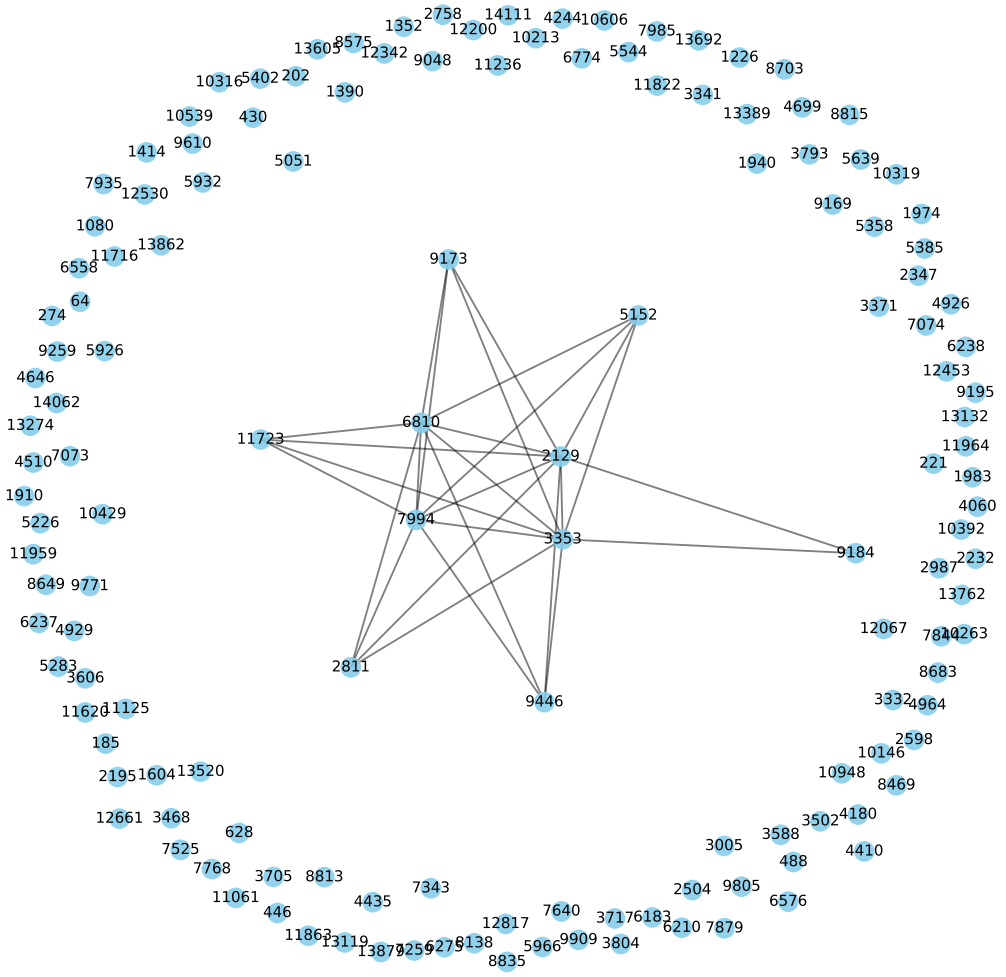


Figura 9. Gráfico de correlación de las 1% dimensiones más activas en Rk.

E. Visualización de la estructura de correlación de las dimensiones.

La Figura 9 visualiza el grafo de correlación descrito en la Sección 4.2.3. En este grafo, cada nodo representa una de las 1% dimensiones más frecuentemente activadas, y se dibuja una línea entre dos nodos si su correlación de Pearson excede 0.9. A partir de la figura, podemos observar claramente una estructura agrupada, lo que sugiere que algunas dimensiones tienden a activarse de manera consistentemente conjunta. Particularmente, dentro del grupo, identificamos un “clique” formado por las dimensiones {2129, 3353, 6810, 7994}, lo que indica que estas dimensiones controlan conjuntamente aspectos de la aleatoriedad general.

F. Estudio de caso.

La Figura 10 muestra cómo las dimensiones en Rk controlan información específica. Se destacan en rojo las diferencias entre las salidas controladas y las soluciones originales. Los conjuntos de dimensiones son: A = {199, 3505, 5283, 7879, 9259, 9468, 9485, 11725}, B = {4060}, C = {1379, 1583, 5475, 7017, 7074, 7521, 8683, 9635, 10460, 10539, 11716, 12012, 13337}, D = {1226}.

Como se muestra en la Sección 4.2.3, ciertas dimensiones influyen en la profundidad del razonamiento, como las A y C. La diferencia entre estas dos dimensiones puede deberse a que la dimensión A afecta principalmente la ecuación final, mientras que la dimensión C influye en la ecuación previa. Esto indica que la información almacenada en las variables aleatorias va más allá del estilo lingüístico superficial, y de hecho puede controlar diversas vías de razonamiento. Otras dimensiones controlan aspectos más detallados de las soluciones, como la expresión explícita de unidades (B) o el formato numérico (D).

Case 1: Raphael went to buy some school supplies. He bought 4 pens which cost $1.5 each, 2 notebooks which cost $4 each, and a rim of bond paper which cost $20. How much did Raphael spend on everything?	**Original Solution:** *Step1:* Raphael spent 4 x $1.5 = $6 on pens. He spent 2 x $4 = $8 on notebooks. *Step2:* Therefore, he spent a total of $6 (for pens) + $8 (for pens) + $20 (for notebooks) = $34. *Step3:* #### 34.	**$R_k(A) = 0.5$:** *Step1:* Raphael spent 4 × $1.5 = $6 on pens He spent 2 × $4 = $8 on notebooks. In total, he spent $6 + $8 + $20 = $34. *Step2:* Therefore, he spent a total of $34. *Step3:* #### 34.	**$R_k(B) = 0.1$:** *Step1:* Raphael spent 4 pens × $1.5/pen = $6 on pens. He spent 2 notebooks × $4/notebook = $8 on notebooks. *Step2:* Therefore, he spent a total of $6 + $8 + $20 = $34 for the items. *Step3:* #### 34.
Case 2: Julie had $500. She spent 20% of it on clothes and then 25% of the remaining money on CDs. How much money did Julie have left?	**Original Solution:** *Step1:* Julie spent $500 * 20/100 = $100 on clothes. She had $500 - $100 = $400 left. *Step2:* She then spent $400 * 25/100 = $100 on CDs. So, she had $400 - $100 = $300 left. *Step3:* #### 300.	**$R_k(C) = 0.5$:** *Step1:* Julie had $500 * 20/100 = $100 left after buying clothes. She had $500 - $100 = $400 left after buying CDs. She then spent 25/100 * $400 = $100 on CDs. *Step2:* She had $400 and she spent $100 so she has 400-100 = $300 left. *Step3:* #### 300.	**$R_k(D) = 0$:** *Step1:* Julie spent $500 * 0.20 = $100 on clothes. She had $500 - $100 = $400 left. *Step2:* She then spent $400 * 25/100 = $100 on CDs. So, she had $400 - $100 = $300 left. *Step3:* #### 300.

Figura 10. Estudio de caso sobre el control de cuatro conjuntos de dimensiones en Rk (k = 1, ..., K). Para cada caso, se destaca la diferencia en la salida del modelo después de la intervención, resaltada en rojo.

Tabla 7. Precisión en la decodificación de NAR en la etapa de habla.

Method	GSM8K	SVAMP	MultiArith
MarCos (text)	9.02	14.96	8.33
MarCos (equation)	14.25	22.82	30.12

G. Limitaciones y Direcciones Futuras

A continuación, se presentan algunas limitaciones en este trabajo, que también señalan posibles vías para futuras investigaciones. En primer lugar, los experimentos presentados en este artículo se centran principalmente en tareas de razonamiento matemático. Esta elección se justifica por el hecho de que el razonamiento basado en transformadores ha demostrado ser más efectivo en este ámbito (Sprague et al., 2025), y la mayoría de los modelos de razonamiento existentes también se evalúan en escenarios matemáticos. En segundo lugar, nuestra evaluación se realiza con problemas matemáticos relativamente básicos, en lugar de con conjuntos de datos más complejos como MATH o AIME. Por un lado, el objetivo de este artículo es demostrar la eficacia y el potencial de MarCos como una estructura de razonamiento continuo, en lugar de buscar resultados orientados a liderboards. Por otro lado, abordar problemas matemáticos complejos podría requerir técnicas adicionales, como el aprendizaje por refuerzo, que dejamos para trabajos futuros. Finalmente, como se discute en la Sección 4.2.1, el "pensamiento latente" representa un nuevo paradigma para los transformadores. Para aprovechar plenamente su potencial, necesitamos explorar métodos y datos de preentrenamiento correspondientes, permitiendo que el modelo adquiera capacidades más amplias y robustas en diversos dominios.

Más allá de estas limitaciones, nuestro estudio también abre varias vías para futuras investigaciones. En primer lugar, y lo más importante, es desarrollar una metodología de pre-entrenamiento para MarCos. El principal desafío reside en la definición y segmentación de los pasos de razonamiento, ya que estos grandes corpus no estructurados carecen de anotaciones paso a paso explícitas. Una posible solución sería tratar cada oración como un único paso de razonamiento. Con esta configuración, el objetivo de pre-entrenamiento puede formularse: dado el conjunto de oraciones anteriores (análogo a la pregunta de nuestro artículo), el modelo debe generar las siguientes K oraciones. Esto es compatible con nuestro esquema de entrenamiento actual, que podría ampliarse. En segundo lugar, como se demuestra en la Sección 4.2.3, nuestra variable aleatoria permite un control a nivel de paso sobre la estocasticidad, ofreciendo una nueva perspectiva para la exploración en el aprendizaje por refuerzo. A diferencia de la muestreo a nivel de token convencional, nuestro enfoque permite muestrear las vías de razonamiento a un nivel superior, lo que podría hacer la exploración más eficiente. En tercer lugar, nuestro marco demuestra que la aleatoriedad puede simularse “una sola vez” en lugar de modelarse de forma iterativa, lo que podría inspirar nuevas direcciones para los modelos de lenguaje basados en difusión. Por último, pero no menos importante, nuestro trabajo demuestra la viabilidad de separar el pensamiento del habla, sugiriendo que el habla puede ser tratado como un proceso de traducción puro. Por lo tanto, sería interesante explorar la introducción de técnicas de decodificación más eficientes, especialmente la decodificación NAR, para acelerar aún más la inferencia.

H. Declaración sobre el uso de LLMs

Utilizamos modelos de lenguaje (LLMs) para mejorar la claridad, la gramática y la fluidez de nuestro artículo. Los LLMs no participaron en ninguna idealización técnica, derivación o generación de contenido de investigación. Hemos revisado cuidadosamente todo el contenido generado por los LLMs para asegurar que no se introdujeron errores factuales o técnicos, y que la integridad científica del trabajo se mantiene por completo.